



문제를 집요하게 파고들어, 운영 가능한 시스템으로 해결하는 백엔드 개발자 안재현입니다.

다양한 초기 창업팀에서 MVP의 기획·개발·출시를 반복하며, 불명확한 요구를 빠르게 구조화하는 역량을 길러왔습니다.

PROFILE

EDUCATION 가천대학교	LOCATION 서울시
MAJOR 인공지능전공 · 창업학 복수전공	FOREIGN LANGUAGE Japanese

CORE CAPABILITIES

문제 구조화

불명확한 요구 → API · 데이터 흐름 · 운영
시나리오

구축과 운영

백엔드 구현 → 배포 · 관측 · 부하 검증

제약 기반 의사결정

일정 · 예산 · 예상 규모 → 기술과 구현 범위

TECHNOLOGY

Backend	Python · FastAPI · Java · Spring Boot	Infrastructure	Kubernetes · Docker · Terraform · Helm · Nginx · Argo CD
Cloud & Ops	AWS · Kakao Cloud · GCP · Prometheus · Grafana · k6	Data	MySQL · PostgreSQL · Redis · MongoDB
Additional	React · TypeScript · Swift		

CONTACT

GITHUB

github.com/Ohjackson

WEBSITE

cantoo.me

EMAIL

wogus3673@naver.com

EXPERIENCE

OIT Cyber Life Design Laboratory

URI-TOMO — 한일 실시간 통역·자막 회의 서비스

2025.12.23–2026.02.04 · Research Intern · Backend

일본 현지에서 백엔드와 Realtime 처리 구조를 설계하고 MVP 데모 완성

Plaid Inc.

B2B AI 서비스 프로토타입 개발

2025.03–2025.08 · Developer

RAG 기반 AI 상담 서비스와 사업 아이디어 검증 프로토타입을 개발하며 초기 서비스 검증에 참여

Acon Inc.

취향 기반 맛집 탐색 서비스

2025.01 · iOS Developer · 초기 창업팀

iOS 클라이언트 개발을 담당하고 초기 버전 출시까지 참여

Wiro · Daon

멘탈 케어 서비스 MVP 개발

2024.08–2024.11 · Full-stack Developer · 초기 창업팀

감정 기록 중심 서비스에서 AI 대화 기반 조기 위험 감지 서비스로 제품 방향을 발전시키며 두 차례의 MVP 개발

INDEPENDENT PRODUCT

단고초

일본어 자동 학습 서비스

2024.06–운영중

학습 과정의 반복 입력을 자동화하는 서비스로, 문제 정의부터 기획·개발, 지속적인 업데이트까지 진행

PROJECTS

01

INFRASTRUCTURE

COCO

04—07

PDF 문서 관리와 지식베이스 기반 AI 질의응답을 연결한 기업 내부 지식 서비스의 인프라 구축·운영 검증

ROLE

Infrastructure Engineer

PERIOD

구축 2주 · 운영 검증 1주

CONSTRAINT

예산 20만 원 · 사용 비용 약 7만 원

VALIDATION

DB 연계 API 4종 · k6 혼합 부하

02

BACKEND

URI-TOMO

08—09

한국어와 일본어가 함께 쓰이는 회의를 위한 실시간 번역·음성 출력·회의 요약 서비스의 백엔드 설계

ROLE

Tech Lead · Backend

TEAM

한·일 7인 협업 프로젝트

REALTIME

LiveKit · Redis · WebSocket

CORE CHANGE

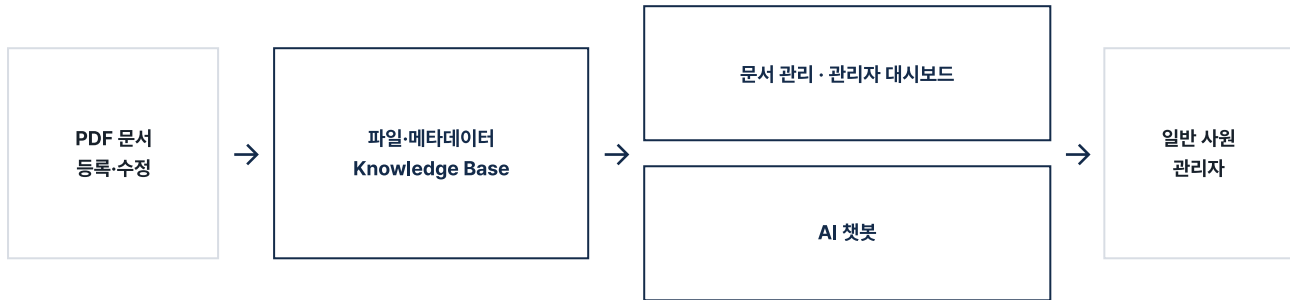
언어별 STT 입력 채널 분리

PROJECT OVERVIEW

COCO

SERVICE

COCO는 조직의 PDF 문서를 등록·관리하고, 축적된 문서를 지식베이스로 연결해 AI 질의응답에 활용하는 기업 내부 지식 서비스입니다. 일반 사원은 문서와 AI 챗봇을 이용하고, 관리자는 같은 시스템에서 문서 현황과 운영 지표를 확인합니다.



문서 관리 화면과 AI 챗봇은 순차 기능이 아니라, 동일한 문서 데이터와 지식베이스를 각 사용 목적에 맞게 제공하는 두 개의 서비스 경로로 설계했습니다.

CONSTRAINTS & PROPOSED DIRECTION

SCHEDULE 3주 인프라 구축 2주 운영·부하 검증 1주	BUDGET 20만 원 고정비를 줄이고 필요한 컴퓨팅에 집중	→	PROPOSE 제한된 기간과 예산 안에서 비용을 최소화하면서, 안정성을 검증하고 이후 최적화할 수 있는 운영 구조를 목표로 설정했습니다.
--	--	---	--

TWO-STAGE INFRASTRUCTURE DECISION

먼저 서비스를 단일 VM에 함께 올릴지 Kubernetes로 분리할지 결정하고, Kubernetes를 선택한 뒤에는 관리형 서비스와 직접 구축 방식의 비용·운영 책임을 비교했습니다.

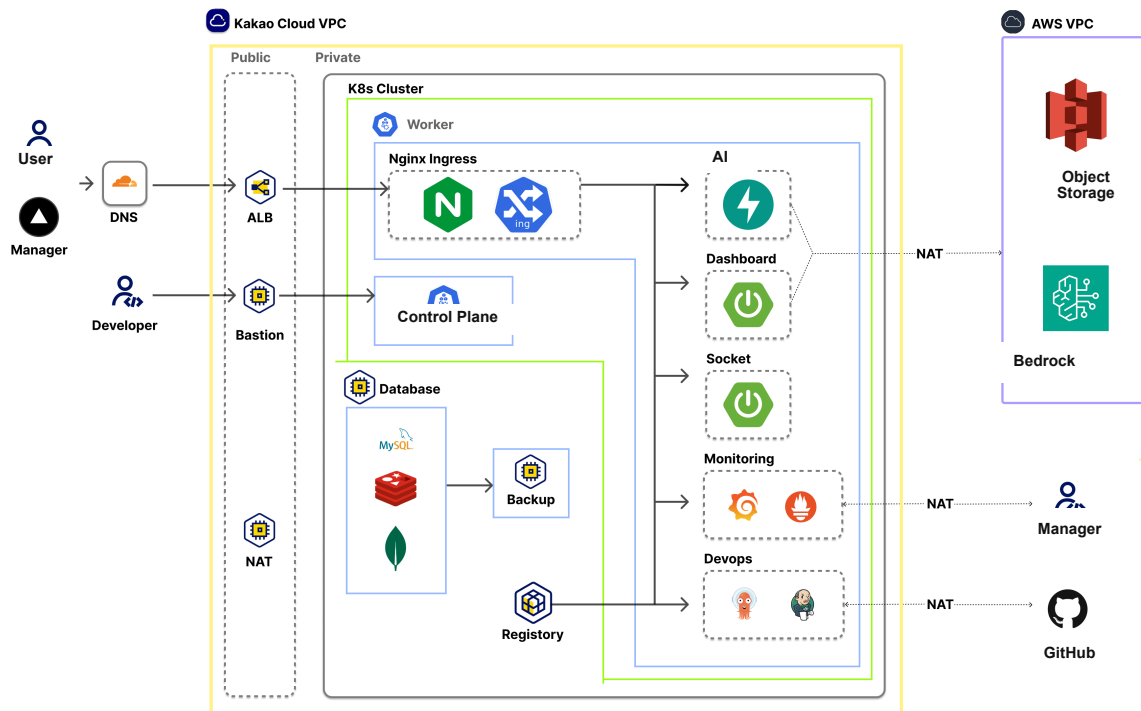
DECISION 1 운영 단위	Single VM →	Kubernetes	Dashboard·Socket·AI API를 분리 배포하고, 필요한 서비스만 Replica 단위로 확장하기 위해 선택했습니다.
DECISION 2 구축 방식	Managed K8s →	kubeadm	짧은 검증 기간에 관리형 서비스의 고정 비용을 부담하기보다, VM 비용 중심으로 예산을 통제하기 위해 선택했습니다.

Dashboard·Socket·AI API를 각각 배포하고 필요한 서비스만 확장할 수 있도록 Kubernetes로 분리했습니다. 3주간 약 7만 원 안에서 운영 구조를 검증하기 위해 관리형 Kubernetes 대신 kubeadm으로 클러스터를 직접 구성했습니다.

INFRASTRUCTURE ARCHITECTURE

COCO

Kakao Cloud VPC를 Public-Cluster-Data 영역으로 나누고, Control Plane-Worker-Database-Backup을 역할별 VM에 배치했습니다. 서비스 트래픽은 DNS → ALB → Nginx Ingress로 전달하고, PDF 저장과 AI 질의응답은 S3-Bedrock과 연계했습니다.



IMPLEMENTED COMPONENTS

Network	Public-Cluster-Data 3개 네트워크 영역 분리, Control Plane-Worker-Database 역할별 배치
Security	cert-manager 기반 TLS 인증서 발급-갱신, Bastion 중심의 관리 접근 경계 구성
Kubernetes	kubeadm 기반 클러스터 구성, Nginx Ingress를 통한 서비스별 라우팅
CI/CD	Jenkins 빌드와 Argo CD 기반 GitOps 배포 흐름 구성
Observability	Prometheus-Grafana-Alertmanager 기반 메트릭 수집-시각화-알림 구성

서비스 초기 이용자를 1,000명으로 가정했습니다. 일반 업무 시간의 반복 요청과 피크 시간 최대 80% 이용을 시나리오로 설정한 뒤, 사용자 응답부터 애플리케이션·서버 자원·데이터 저장까지 함께 확인할 수 있도록 관측 항목을 정의하고 테스트했습니다.

OBSERVATION LAYERS

01 · USER EXPERIENCE 요청이 정상적으로 끝나는가 처리량, p95 응답시간, HTTP 오류율과 Timeout을 함께 확인해 사용자가 체감하는 현상을 먼저 정의했습니다.	02 · APPLICATION 애플리케이션이 안정적인가 Replica 상태, Pod Restart, OOMKilled와 로그를 확인해 배포 실패와 런타임 장애를 구분했습니다.
03 · RESOURCE 자원이 포화되고 있는가 CPU 사용량과 Throttling, 메모리와 Node 상태를 비교해 단순 지연과 자원 한계를 분리했습니다.	04 · DATA 쓰기와 저장이 보장되는가 DB 오류, Connection, 쓰기 실패와 데이터 정합성을 확인해 API 응답 뒤의 저장 결과까지 추적했습니다.

OPERATION LOOP

코드 변경이 운영 환경에 반영된 뒤 지표와 로그를 확인하고, 이상이 발생하면 배포·애플리케이션·자원·데이터 중 원인이 있는 구간부터 대응하는 흐름으로 정리했습니다.

코드 변경	테스트·이미지	Argo Sync	K8s 배포	지표·로그 관측	원인별 대응
-------	---------	-----------	--------	----------	--------

요청 결과와 Pod 상태, CPU·메모리, DB 저장 결과를 같은 시간대에 비교해 부하가 증가할 때 오류가 어느 지점에서 시작되는지 확인할 수 있도록 했습니다.

LOAD VALIDATION

COCO

TEST DESIGN

정상 업무 구간을 먼저 확인한 뒤, 피크 시간 이용 가정에 가까운 스트레스 구간까지 부하를 점진적으로 높였습니다. 최대 수치만 제시하기보다 조회와 쓰기 API의 차이, Pod 상태와 DB 처리 결과를 함께 비교하는 데 초점을 두었습니다.

TEST SCENARIO	통계 조회 GET 2종 · PDF 등록 POST 1종 · 문서 관리 UPDATE 1종을 혼합했습니다. AI 응답 경로는 외부 모델 지연이 서버 용량 판단을 왜곡하지 않도록 분리했습니다.		
LOAD PROFILE 10 → 170 → 800 VU	DURATION 각 5분 · 총 15분	WORKLOAD 4 API 동시 호출 · 1초 간격	ENVIRONMENT Backend 2 Replicas · 1 Worker

VU는 RPS가 아닌 동시 실행 사용자 설정값입니다. HTTP 오류-p95, Pod Restart-OOMKilled, CPU-Throttling과 작업 완료 상태를 함께 비교했습니다.

NORMAL LOAD

170 VU

- Backend 2 Replicas
- 주요 API 혼합 요청 처리
- Pod Restart 0
- OOMKilled 0
- RPS · p95 · HTTP 오류 동시 관측

정상 용량은 서버가 버티는 최대치가 아니라, 응답과 자원 상태를 함께 만족하는 운영 구간으로 정의했습니다.

INITIAL STRESS LOAD

800 VU

- 조회 API 응답 유지
- PDF 등록 경로 HTTP 500 약 5%
- CPU 사용량-Throttling 동반 상승
- Pod Restart · OOMKilled 0
- 쓰기 경로에서만 오류 집중

PROBLEM RESOLUTION

OBSERVE

HTTP 500
약 5%

조회 API는 유지됐지만 PDF 등록 경로에서만 오류가 발생했고, 같은 구간에서 CPU Throttling이 상승했습니다.



NARROW DOWN

CPU 포화보다
쓰기 경로 추적

CPU 사용량의 절대치는 포화 수준이 아니었고 Restart-OOMKilled도 없어, PDF 처리 흐름을 우선 확인했습니다.



TRACE & FIX

작업 큐 → Worker
→ 상태 저장 · 재시도

로그와 작업 상태를 추적해 큐에 등록되지 않거나 종료되지 않는 작업을 확인했습니다. 큐 등록, 성공·실패 상태 저장과 제한된 재시도 흐름을 보완했습니다.



RETEST

500 오류율
5% → 0%

실제 PDF 처리가 정상 수행되면서 업로드 API 응답시간은 약 1초 증가했지만 오류는 재발하지 않았습니다.

Trade-off · 처리시간의 약 1초 증가를 수용하고, 작업 누락과 서버 오류를 제거하는 쪽을 선택했습니다.

3주간 사용 비용 약 7만 원 / 예산 20만 원. 부하 수치 자체보다 지표를 통해 병목 범위를 좁히고, 원인을 수정한 뒤 같은 조건에서 재검증하는 과정에 집중했습니다.

SERVICE

한국어와 일본어가 함께 쓰이는 회의를 위한 실시간 번역·음성 출력·회의 요약 서비스입니다.

CONTEXT

OIT 연구실 의뢰

TEAM

한·일 7인 협업

SCOPE

인증 · 세션 · Realtime

REQUEST & EVENT FLOW

회의 시작 요청이 들어오면 백엔드가 사용자와 참여 정보를 검증하고 LiveKit Room을 준비합니다. 이후 Room Worker가 자동으로 연결되어 STT·번역·TTS를 처리하며, 사용자는 별도로 Worker를 선택하거나 제어하지 않습니다.



BACKEND RESPONSIBILITY

Meeting API	Realtime Worker	Event · Data Path
로그인 사용자와 회의 참여 여부를 검증하고, 회의 시작 요청에 맞춰 LiveKit Room과 미디어 연결 정보를 생성했습니다.	회의가 시작되면 Room Worker가 자동으로 연결되고, 회의방 단위로 음성을 받아 STT·번역·TTS 처리 흐름을 실행하도록 구성했습니다.	Redis와 WebSocket으로 실시간 이벤트를 전달하고, MySQL에 회의 기록과 요약 결과를 남겼습니다.

프론트엔드와 AI 기능을 인증부터 실시간 처리, 회의 기록과 요약까지 이어지는 하나의 백엔드 흐름으로 통합했습니다.

PROBLEM DEFINITION

언어 인식 오류를 모델 문제가 아닌 입력 경계의 문제로 재정의했습니다.



STRUCTURAL CHANGE

언어별 Realtime Session	입력·출력 Audio Track 분리	번역 방향을 시스템에서 명시	모듈별 책임과 연동 규칙 단순화
-------------------------	-------------------------	--------------------	----------------------

IMPLEMENTED RESULT

INPUT ROUTING 언어 속성 기반 세션 자동 할당 참여자의 KO·JA 속성을 확인해 해당 Realtime Session으로 입력을 라우팅했습니다.	OUTPUT PATH KO·JA Audio Track 분리 번역 음성을 언어별 출력 트랙으로 발행해 전달 경로가 섞이지 않도록 구성했습니다.	ERROR BOUNDARY 번역 방향과 언어 메타데이터 명시 모델의 언어 추정 단계를 제거해 STT 오류가 반대 언어 출력으로 전파되는 경로를 줄였습니다.
--	---	---

DEMO RESULT · MVP 데모에서 한·일 발화가 언어 속성에 따라 각 Realtime Session으로 라우팅되고, 번역 결과가 언어별 출력 트랙으로 전달되는 흐름을 확인했습니다.

**좋은 하루 보내세요.
감사합니다!**

EMAIL

wogus3673@naver.com

GITHUB

github.com/Ohjackson

WEBSITE

cantoo.me